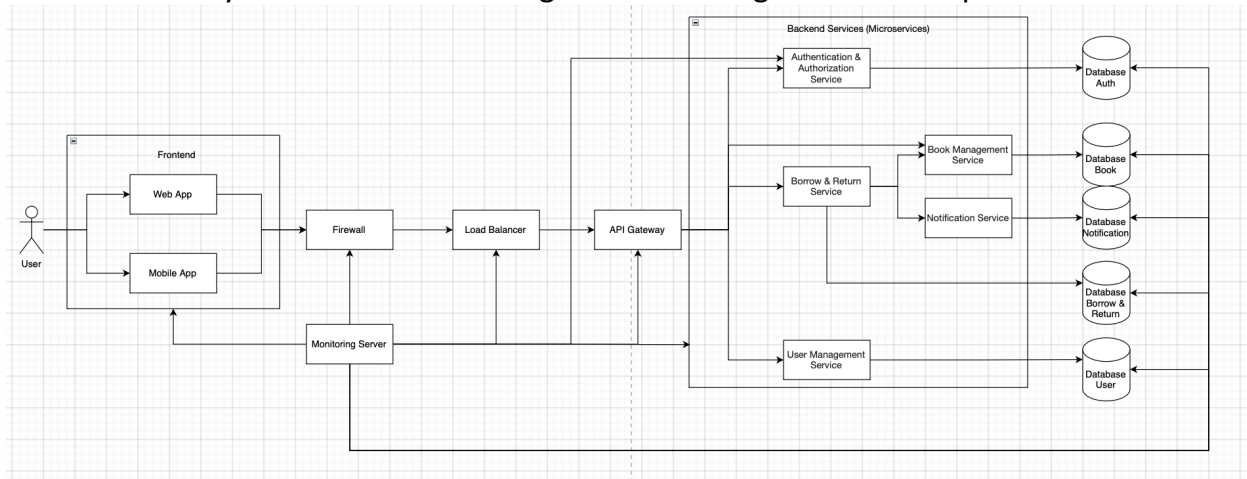


Question 3:

1. Draw a system architecture diagram illustrating how the components interact.



2. Explain the role of each component in the system.
 - **Frontend:** The frontend is what users interact with, whether they are library staff or regular users. It consists of two main parts:
 - **The Web app:** This is used by the staff and library administrator to manage reader, book, monitor system status, and handle administrative tasks.
 - **The Mobile app:** Designed for library users to search for books, make reservations, borrow books via QR/Barcode, and check their personal borrowing history.
 - **Firewall (DMZ):** Protecting the system from external threats and unauthorized access by filtering incoming and outgoing traffic.
 - **Load Balancer:** Distributes incoming network traffic across multiple servers (API Gateways) to ensure no single server has too much load, increasing system availability and reliability.
 - **API Gateway:** The single entry point for all clients. It manages requests, handles routing to appropriate microservices, and can perform tasks like protocol translation and request rate limiting.
 - **Backend Services (Microservices):**
 - **Authentication & Authorization Service:** Manages user identity, login processes, and Role-Based Access Control (RBAC) to ensure security.
 - **Book Management Service:** Handles all book-related operations such as add, edit, delete, and search the library catalog.
 - **Borrow & Return Service:** Manages the core logic of borrowing and returning books, tracking deadlines, and updating book status.

- **User Management Service:** Manages profile information for both library staff and users.
 - **Notification Service:** Responsible for sending real-time alerts or emails to users regarding due dates, successful reservations, or system updates.
 - **Database:** Following the "Database per Service" pattern, each microservice has its own dedicated database to ensure data isolation, independence, and scalability.
 - **Monitoring Server:** Continuously monitors the health and performance of the entire system, including servers, services, and databases, to provide alerts and logs for troubleshooting.
3. Propose technologies to use for each component in the model.
- **Frontend: ReactJS for Web and Flutter for Mobile** for high performance and ease of use.
 - **Firewall: AWS WAF or pfSense** for strong and reliable system protection.
 - **Load Balancer: Nginx**, a popular and lightweight tool for sharing network traffic.
 - **API Gateway: Kong Gateway** to manage and route API requests efficiently.
 - **Backend Services: Java Spring Boot or Node.js**, which are perfect for building microservices.
 - **Authentication & Authorization: Keycloak or Spring Security** using JWT to verify users and manage permissions.
 - **Database: MySQL or PostgreSQL** to store structured data like books and user info.
 - **Monitoring: Prometheus & Grafana** to collect system data and show it on easy-to-read charts.
 - **Book, Borrow & User Services: .NET Core 8.0** to build high-performance and scalable microservices.
 - **Notification Service: Node.js & RabbitMQ** to handle asynchronous tasks and send alerts efficiently.
 - **Message Queue: RabbitMQ or Kafka** to help different services communicate without slowing down the system.
 - **Push Notification: Firebase (FCM)** to send real-time notifications to both Web and Mobile users.
 - **Logging: ELK Stack** to collect and manage all system logs in one central place for easy debugging.